

Técnicas de Diseño de Algoritmos
Primer Cuatrimestre 2025
1er Parcial

Nombre y Apellido	L.U.	N°Orden
Resuelto		

Turno: ☐ Mañana ☐ Tarde/Noche Materia: ☐ TDA ☐ AED 3

Duración: 5 horas. Este examen es a libro cerrado e individual. La nota de aprobación es de 60/100 puntos.

1. Preguntas de opción multiple

Cada pregunta tiene 4 opciones y una sola opción correcta. Las respuestas correctas (Bien) suman 5 puntos. Las respuestas incorrectas (Mal) restan 2 puntos. Las preguntas no respondidas no se consideran como incorrectas y suman 0 puntos. Marcar en el cuadrado la opción que consideren correcta.

Pregunta 1 Sea la siguiente función recursiva que calcula el número combinatorio $\binom{N}{K}$

$$\binom{N}{K} = \begin{cases} 1, & \text{si } k = 0 \vee k = n \\ 0, & \text{si } k < 0 \vee k > n \\ \binom{N-1}{K-1} + \binom{N-1}{K} & \text{si no} \end{cases}$$

¿Cuál es la complejidad temporal de calcular el número $\binom{N}{K}$ utilizando Programación Dinámica?

☐ $\Theta(N^2)$

☐ $\Theta(K^2)$

☒ $\Theta(N * K)$

☐ $\Theta(N^2 * K)$

Pregunta 2 Dado un grafo con N nodos y M aristas. ¿Cuál es la complejidad espacial de la representación como matriz de adyacencia?

☒ $\Theta(N^2)$

☐ $\Theta(N + M)$

☐ $\Theta(N * M)$

☐ $\Theta(M)$

Pregunta 3 Si en la representación de un grafo de N nodos (1 a N) y M aristas como lista de adyacencia, reemplazo los vectores que contienen a los vecinos de cada nodo por tablas hash (*) (ej: unordered_set en C++, HashSet en Rust, dict en Python), podré:

☐ Poder recorrer los vecinos de un nodo v dado en orden ascendente en $O(d(v))$

☐ Tener complejidad espacial $O(N)$

☒ Saber si dos nodos u y v son vecinos en $\Theta(1)$

☐ Saber si dos nodos u y v tienen vecinos en común en $O(\log(N))$

(*) Durante el parcial no se responderán consultas sobre lo que es una tabla hash.

Pregunta 4 Dado el dígrafo con 5 nodos y aristas: (1, 2), (1, 3), (2, 4), (3, 4), (4, 5). ¿Cuál de los siguientes es un recorrido de DFS válido?

☒ 1 - 2 - 4 - 5 - 3

☐ 1 - 2 - 4 - 3 - 5

☐ 1 - 2 - 3 - 4 - 5

☐ 1 - 3 - 2 - 4 - 5

Pregunta 5 Dado un grafo dirigido con nodos del 1 al N , donde el nodo i ($1 \leq i \leq N$) tiene asociado un string s_i (todos los s_i son no vacíos y distintos) en el cual hay una arista (i, j) si y solo si s_j es un sufijo de s_i . ¿Cuál de las siguientes afirmaciones es **falsa**?

- ☐ De un nodo i pueden salir a lo sumo $|s_i|$ aristas
- ☐ El grafo tiene bucles (pares (i, i))
- ☐ El grafo es transitivo
- ☒ El grafo puede tener ciclos dirigidos simples con más de un nodo

(*) $|s|$ es la cantidad de caracteres de s

Pregunta 6 Si represento a una función recursiva f (que al computarla no genera una recursión infinita) como un grafo dirigido donde cada uno de los N nodos es un estado y una arista (a, b) significa “La función se llama a sí misma con el estado b para procesar el estado a ”. ¿Cuál de las siguientes afirmaciones es correcta?

- ☐ Puede haber ciclos en el grafo
- ☐ Siempre podemos calcular la función f para todos los estados en $O(N)$
- ☐ Los casos base tienen grado de entrada 0
- ☒ Llamar a la función con el estado x generará tantas llamadas al estado y como caminos hay de x a y en el grafo

Pregunta 7 La relación entre los nodos de un grafo no dirigido definida como $R(a, b)$ si y sólo si a y b pertenecen a la misma componente conexa, es:

- ☒ Una relación de equivalencia
- ☐ Anti-simétrica ($R(a, b) \implies \neg R(b, a)$)
- ☐ Una relación de orden (parcial o total)
- ☐ Una relación irreflexiva (no existe $R(a, a)$)

Pregunta 8 La complejidad espacial de la memorización de una DP con N estados y M transiciones entre estados, implementada de forma top-down recursiva, es:

- ☐ $\Theta(M)$
- ☐ $\Theta(N + M)$
- ☒ $\Theta(N)$
- ☐ $\Theta(N * M)$

2. Preguntas abiertas de respuesta corta

Respuesta Bien: +5 puntos. Respuesta Regular, Mal o sin completar: +0 puntos.

Pregunta 9 En el algoritmo de DP de Floyd Warshall visto en clase, ¿Qué se calcula en cada estado de la DP?

El estado (k, i, j) calcula el camino mínimo de i a j que solo usa a nodos en el conjunto $1..k$ como nodos intermedios.

Pregunta 10 En el algoritmo de DP de Bellman Ford visto en clase. ¿Qué se calcula en cada estado de la DP?

El estado (L, i) calcula el camino mínimo desde el origen hasta i que tenga exactamente L aristas.

Pregunta 11 Sea G un grafo dirigido (dígrafo) donde los nodos son los números de 1 a N y existe una arista (a, b) si y sólo si existe un número primo p tal que $b = a * p$. ¿Qué significa que exista un camino de u a v en G ?

Significa que v es un múltiplo de u

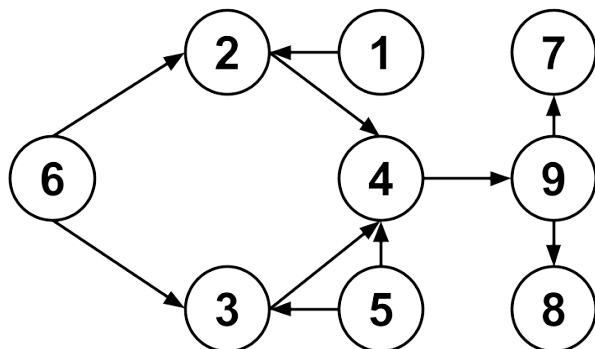
Técnicas de Diseño de Algoritmos
Primer Cuatrimestre 2025
1er Parcial

Nombre y Apellido	L.U.	N°Orden
.....		

Turno: ☐ Mañana ☐ Tarde/Noche Materia: ☐ TDA ☐ AED 3

Duración: 5 horas. Este examen es a libro cerrado e individual. La nota de aprobación es de 60/100 puntos.

Pregunta 12 Indique un orden topológico válido del siguiente grafo dirigido acíclico:



1 - 5 - 6 - 2 - 3 - 4 - 9 - 7 - 8

3. Ejercicios a Desarrollar

Ejercicio Bien: +10 puntos. Ejercicio Regular: +5 puntos. Ejercicio Mal o sin completar: +0 puntos.

Pregunta 13 Dado un grafo dirigido $G = (V, E)$ con nodos números de 1 a N . Se tiene asociado a cada nodo v un peso p_v entero positivo.

Se pide diseñar un algoritmo o utilizar uno existente que en $O(N^3)$ (espacial y temporal) determine cada par ordenado de nodos (u, v) si es posible llegar de u a v en el dígrafo dado y, si es posible, determinar el menor peso P tal que es posible llegar utilizando como nodos intermedios (sin contar u ni v) solo a nodos con un peso menor o igual a P .

En caso de utilizar una modificación a un algoritmo existente, solo es necesario detallar cuales serían estas modificaciones.

Utilizamos el algoritmo de Floyd-Warshall con las siguientes modificaciones/consideraciones:

- $dp[0][i][j] = -\infty$ si $i = j$ o si existe $i -> j$
- $dp[0][i][j] = \infty$ en caso contrario
- Reemplazo la actualización del estado por $dp[k][i][j] = \min(dp[k-1][i][j], \max(dp[k-1][i][k], p_k, dp[k-1][k][j]))$

De esta forma, en la matriz final:

- $dp[N][i][j] = -\infty$ significa que existe un camino de i a j sin nodos intermedios.
- $dp[N][i][j] = \infty$ significa que no existe un camino de i a j
- En caso contrario, el P para el par (i, j) es $P = dp[N][i][j]$

Pregunta 14 Dado un cubo de Rubik, definimos el estado del Cubo como una posición definida por el color de cada celda que hay en cada cara (cada cara tiene un identificador que permite diferenciarlas). El cubo se considera resuelto si cada cara tiene únicamente celdas de un mismo color. Una jugada válida consiste en rotar una cara del cubo 90 grados en sentido horario o anti-horario.

Se pide modelar el cubo con un grafo donde:

- A) Cada nodo represente un estado del cubo.
- B) La distancia entre dos nodos u y v sea igual a la cantidad de jugadas necesarias para convertir el estado representado por u en el estado representado por v .

En este caso, el modelo es simple:

- Como se dice en la consigna, cada nodo v del grafo es un estado del cubo válido.
- Existe una arista (u, v) si es posible pasar de un estado al otro en un movimiento válido.

El grafo no es ponderado ni dirigido.

Pregunta 15 Sea el problema Sudoku N el problema de rellenar una matriz de $N \times N$ con números de 1 a N tal que no haya dos celdas con el mismo número en la misma fila ni en la misma columna. Sean estas dos soluciones con Backtracking a dicho problema:

```

1 def Sudoku(estado: list[list[int]], celda: int) -> int:
2     if celda == N * N:
3         return int(es_valido(estado))
4     fil, col = celda // N, celda % N
5     retorno = 0
6     for num in range(1, N+1):
7         estado[fil][col] = num
8         retorno += Sudoku(estado, celda + 1)
9         estado[fil][col] = 0
10    return retorno

```

```

1 def Sudoku_poda(estado: list[list[int]], celda: int) -> int:
2     if celda == N * N:
3         return 1
4     fil, col = celda // N, celda % N
5     retorno = 0
6     for num in range(1, N+1):
7         estado[fil][col] = num
8         if es_valido(estado):
9             retorno += Sudoku_poda(estado, celda + 1)
10            estado[fil][col] = 0
11    return retorno

```

Demostrar que:

- A) La cantidad de estados visitados por Sudoku es $\Theta(N^{(N^2)})$
- B) La cantidad de estados visitados por Sudoku_poda es $O((N!)^N * N^2)$

Puede usar lo siguiente sin necesidad de demostrarlo:

$$\sum_{i=1}^{i=n} p^i \in \Theta(p^n)$$

Cada una de las 2 demostraciones puede estar Bien, Regular o Mal. Para tener el ejercicio Bien ambas deben estar Bien. Para tener el ejercicio Regular se necesita que ambas estén Regular o al menos una esté Bien. Caso contrario, se tendrá el ejercicio Mal.

Item A

Notar que cada estado consiste en:

- Tomar un prefijo de longitud i de la matriz de $N \times N$. Es decir, $0 \leq i \leq N^2$
- Rellenar cada una de sus posiciones con un elemento entre 1 y N

De esta forma, para un prefijo de longitud i hay N^i formas de llenarlo. Por tanto, la cantidad total de estados es:

$$\sum_{i=0}^{N^2} N^i \in \Theta(N^{(N^2)}) \quad (\text{uso sin demostrar por la consigna})$$

Item B

Notar que ahora cada estado es un prefijo de una solución válida:

- En una solución válida, cada fila es una permutación de los N números.
- Existen $N!$ posibles permutaciones
- La cantidad de soluciones válidas está acotada por $(N!)^N$ (producto cartesiano de las filas)
- Cada estado se conforma de un índice (N^2 posibilidades), que determina la longitud del prefijo, y una solución válida ($(N!)^N$ posibilidades), dando por producto cartesiano $O((N!)^N * N^2)$ posibles estados (notar que es una cota superior)

Pregunta 16 Sea el problema:

Dado un conjunto S de N items (p_i, v_i) y un peso límite P , no negativo. Determinar el máximo valor posible V tal que existe un subconjunto S' de S tal que:

A) La suma de los p_i con i en S' es menor o igual a P .

B) La suma de los v_i con i en S' es igual a V .

Dicho de otra forma, el máximo valor total entre los subconjuntos cuyo peso total no excede P .

Y dada la siguiente solución (S es dado como una lista):

```

1 def Mochila(P: int, S: list[tuple[int,int]]) -> int:
2     if P < 0: return -inf
3     if len(S) == 0: return 0
4     (peso, valor) = S[0]
5     return max(Mochila(P, S[1:]), Mochila(P-peso, S[1:])+valor)

```

Demostrar que $Mochila(P, S)$ (con los P y S del enunciado original) devuelve la respuesta al problema planteado, independientemente del orden de los elementos en S .

Utilizo inducción:

La hipótesis inductiva es que si $Mochila(P', S')$ se calcula correctamente para todo estado tal que $P' \leq P \wedge S' \subsetneq S$, entonces se calcula correctamente para (P, S) . Evaluemos los distintos casos:

- Caso base: $P < 0$. Si se me permite un peso negativo, es imposible dar una solución. Por tanto, está bien devolver $-\infty$
- Caso base: Si S es vacío, la única solución es no tomar ningún item, lo que tiene valor 0.
- Caso recursivo: Sea x un elemento de S (en el código $S[0]$). Podemos dividir los subconjuntos de S entre los que contienen a x y los que no contienen a x .

En el caso recursivo, evalúo las únicas dos posibilidades:

Para esto, defino $S' = S - \{x\}$

- Si la respuesta no utiliza x , entonces será igual al máximo posible con el mismo peso P y con los elementos de S' . Por **hipótesis inductiva**, este valor se calcula correctamente al llamar a $Mochila(P, S')$
- Si la respuesta sí utiliza x , entonces consistirá en reservar espacio para x , con el espacio restante y S' obtener el mayor valor posible, y finalmente sumar el valor de x . Esto se hace en $Mochila(P - peso, S') + valor$. Nuevamente, este valor está calculado correctamente por **hipótesis inductiva**.

Por tanto, usando la **hipótesis inductiva** llego a que calculo tanto el mejor valor posible utilizando x como sin utilizarlo. Lógicamente, el mejor valor posible para el caso (P, S) es el máximo de estos dos valores.